

Final Technical Report

MacraScript: A Domain-Specific Language
for Macramé Pattern Generation

Juan Manuel Serrano Rodriguez - 20211020091
Javier Alejandro Sánchez Salamanca - 20201020167
Computer Science 3

Universidad Distrital Francisco José de Caldas

May 12, 2025

Contents

Abstract	2
1 Introduction	3
1.1 Problem Context	3
1.2 Project Objectives	3
1.3 Motivation and Target Audience	3
2 Final Language Design	5
2.1 Overview	5
2.2 Alphabet and Tokens	5
2.3 Final Grammar	5
2.4 Syntax Examples	6
2.4.1 Alpha Pattern Example	6
2.4.2 Normal Pattern Example	7
3 System Architecture and Implementation	8
3.1 Compiler Pipeline	8
3.2 Graphical User Interface (<code>compiler.py</code>)	8
4 Results and Discussion	10
4.1 Graphical Outputs	10
4.1.1 Pixel Art Visualization	10
4.1.2 Knot-Flow Diagram	10
4.2 Conclusion	12
5 Design Rationale and Reflection	13
5.1 Language Design Choices	13
5.2 Compiler and Visualization Choices	13
6 Error Handling	14
6.1 Lexical Errors	14
6.2 Syntactic Errors	15
6.3 Semantic Errors	15
Bibliography	15

Abstract

This technical report details the design, implementation, and final results of the MacraScript project. MacraScript is a domain-specific language (DSL) created to address the lack of specialized digital tools for designing and visualizing macramé patterns. The project involved the development of a complete compiler in Python, featuring lexical, syntactic, and semantic analyzers, along with a Tkinter-based graphical user interface. The language supports two distinct pattern types, "Alpha" and "Normal," and generates dual graphical outputs for each: a top-down pixel-art preview and a detailed, step-by-step knot-flow diagram. This document covers the final language grammar, the complete architecture of the compiler, the logic behind the graphical rendering, and the results achieved, demonstrating a functional and effective solution that bridges the gap between traditional crafting and computational design.

Chapter 1

Introduction

1.1 Problem Context

Macramé is a textile art form based on creating decorative patterns through organized knotting sequences. The planning of complex patterns is a meticulous process, prone to human error and lacking specialized digital tools. Artisans often rely on hand-drawn grids or generic design software, which are not optimized for the specific constraints of knot-based patterns [3].

1.2 Project Objectives

The primary goal of this project was to develop MacraScript, a DSL that allows designers to:

- Define macramé patterns using a simple, declarative syntax.
- Specify parameters such as thread count, colors, and knot sequences.
- Generate accurate visualizations of the final pattern.
- Produce step-by-step graphical guides for executing the pattern.

1.3 Motivation and Target Audience

The development of MacraScript is driven by the need to provide a specialized tool for a community that has largely been overlooked by modern software development. The primary beneficiaries of this language are:

- **Macramé Artisans and Designers:** Professionals and hobbyists who can now create, verify, and iterate on complex patterns with greater speed and accuracy. MacraScript reduces the tediousness of manual drafting and prevents costly errors in the middle of a physical project.
- **Educators and Learners:** The language serves as an educational tool, helping newcomers understand pattern construction through a logical, structured format. The dual visualization provides a clear link between the abstract code and the physical knotting process.

- **Digital Crafters:** A growing community of makers who merge traditional crafts with digital tools. MacraScript provides a platform for experimentation, sharing, and archiving patterns in a standardized, reproducible format.

By creating a bridge between computational thinking and textile art, MacraScript not only streamlines a creative process but also fosters a new way of thinking about and sharing traditional knowledge.

Chapter 2

Final Language Design

2.1 Overview

MacraScript is a declarative DSL designed to be intuitive for macramé artisans. The language is structured around a central **PATTERN** block, which can be of type **ALPHA** (for grid-based designs) or **NORMAL** (for knot-based geometric patterns).

2.2 Alphabet and Tokens

The language's alphabet includes uppercase and lowercase letters, numbers, and the symbols `:`, `{ }`, `( )`, `"`. Reserved words, or keywords, are **START**, **END**, **PATTERN**, **ALPHA**, **NORMAL**, **THREADS**, **WIDTH**, **HEIGHT**, **COLORS**, **ROW**, **LEFT**, **RIGHT**.

The lexical analyzer uses the following regular expressions to tokenize the input code:

Table 2.1: Token Definitions and Regular Expressions

Token Type	Regular Expression
KEYWORDS	<code>\b(START END ... RIGHT)\b</code>
COLOR	<code>"#?[A-Fa-f0-9]{6}"</code>
COLOR_NAME	<code>"[a-zA-Z]+"</code>
INTEGER	<code>\d+</code>
COLON	<code>:</code>
COMMA	<code>,</code>
LBRACE	<code>{</code>
RBRACE	<code>}</code>
LPAREN	<code>\(</code>
RPAREN	<code>\)</code>
SKIP	<code>[\t\r\n]+</code>
MISMATCH	<code>.</code>

2.3 Final Grammar

The final grammar for MacraScript is defined in a clear, readable EBNF format. Each production rule is explained to enhance comprehension.

```
1 (* Program Structure: Defines the main container for a pattern *)
```

```

2 <program> ::= "START" <type> <config> <pattern> "END"
3
4 (* Pattern Type: Specifies one of the two supported pattern
   styles *)
5 <type> ::= "ALPHA" | "NORMAL"
6
7 (* Configuration Block: Contains all setup parameters *)
8 <config> ::= <threads> [<dims>] <colors>
9 <threads> ::= "THREADS" ":" <INTEGER>
10 <dims> ::= ("WIDTH" | "HEIGHT") ":" <INTEGER>
11 <colors> ::= "COLORS" ":" "(" <string_list> ")"
12 <string_list> ::= <STRING> ("," <STRING>)*
13
14 (* Pattern Data: The core block defining the pattern's structure
   *)
15 <pattern> ::= "PATTERN" "{" <row_list> "}"
16 <row_list> ::= <row>+
17 <row> ::= "ROW" ":" "(" <sequence> ")"
18
19 (* Sequence Types: Defines the content of a row based on pattern
   type *)
20 <sequence> ::= <alpha_sequence> | <normal_sequence>
21 <alpha_sequence> ::= <INTEGER> ("," <INTEGER>)* (* A list of
   color indices *)
22 <normal_sequence> ::= <DIRECTION> ("," <DIRECTION>)* (* A list of
   knot directions *)
23 <DIRECTION> ::= "LEFT" | "RIGHT"

```

Listing 2.1: Final EBNF Grammar for MacraScript

2.4 Syntax Examples

2.4.1 Alpha Pattern Example

```

1 START ALPHA
2 THREADS: 4
3 WIDTH: 4
4 HEIGHT: 4
5 COLORS: ("red", "blue")
6 PATTERN {
7     ROW: (0, 1, 0, 1)
8     ROW: (1, 0, 1, 0)
9     ROW: (0, 1, 0, 1)
10    ROW: (1, 0, 1, 0)
11 }
12 END

```

Listing 2.2: Alpha pattern for a 4x4 checkerboard

2.4.2 Normal Pattern Example

The syntax for Normal patterns was refined to be more intuitive. Instead of specifying individual knots, the user defines rows of knot directions. The compiler automatically handles the staggered pairing of threads.

```
1 START NORMAL
2 THREADS: 8
3 COLORS: ("red", "blue", "yellow", "green")
4 PATTERN {
5     ROW: (RIGHT, RIGHT, RIGHT, RIGHT)
6     ROW: (LEFT, LEFT, LEFT)
7     ROW: (RIGHT, RIGHT, RIGHT, RIGHT)
8 }
9 END
```

Listing 2.3: Normal pattern with alternating rows

Chapter 3

System Architecture and Implementation

3.1 Compiler Pipeline

The MacraScript compiler is a Python application that processes the source code in three main phases:

1. **Lexical Analysis** (`lexical.py`): This module uses regular expressions to break the input code into a list of tokens. It correctly handles keywords, identifiers, color names, and hex color codes.
2. **Syntactic Analysis** (`sintactic.py`): A recursive descent parser validates the token sequence against the defined grammar. It builds a dictionary representing the abstract structure of the code, which is passed to the next phase.
3. **Semantic Analysis** (`semantic.py`): This is the core of the compiler's logic. It takes the parsed structure and:
 - Validates semantic rules (e.g., thread counts, color indices).
 - For `NORMAL` patterns, it translates the abstract rows of directions into a concrete list of knot instructions, automatically calculating the correct staggered thread pairs for each knot.
 - Produces a high-level `MacraPattern` object that is easy for the rendering engine to consume.

3.2 Graphical User Interface (`compiler.py`)

The main application module uses Tkinter to provide a simple GUI. It includes a text editor for writing MacraScript code and two canvas widgets for displaying the graphical outputs. This module orchestrates the compilation pipeline and calls the appropriate drawing functions. A mockup of the proposed UI is shown in Figure 3.1.

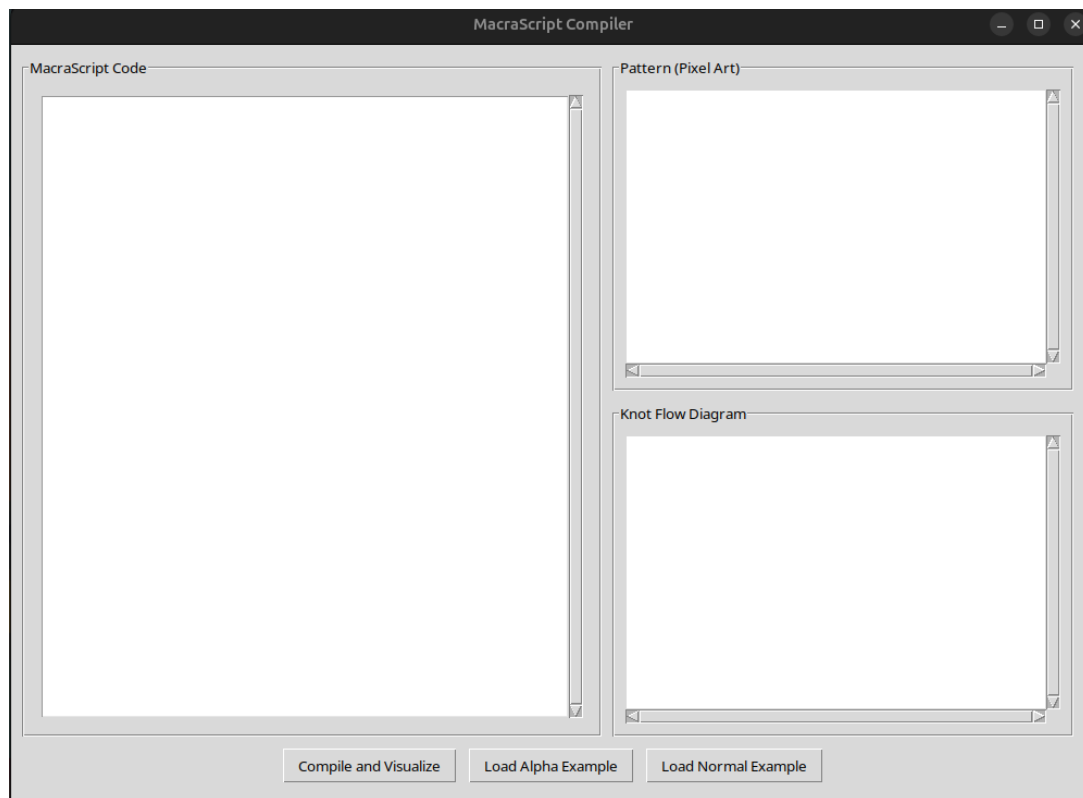


Figure 3.1: The MacraScript IDE, showing the code editor, visualization panels, and a dedicated error console at the bottom.

Chapter 4

Results and Discussion

4.1 Graphical Outputs

The final implementation successfully generates two distinct and coherent visualizations for each pattern type, providing both a final preview and a functional guide.

4.1.1 Pixel Art Visualization

This view shows a top-down preview of the finished piece.

- **Alpha:** Renders a grid of colored squares.
- **Normal:** Renders a grid of colored diamonds. The color of each diamond correctly represents the color of the *active* thread that forms the knot, ensuring visual consistency with the knot diagram.

4.1.2 Knot-Flow Diagram

This view provides a step-by-step guide for the artisan.

- **Alpha:** Renders a snake-like flow of nodes. The connecting lines are colored to match the node they originate from, improving clarity.
- **Normal:** Renders a detailed lattice diagram showing the path of each thread. The knots are represented by circles, colored according to the active thread, with an arrow indicating the knot's direction.

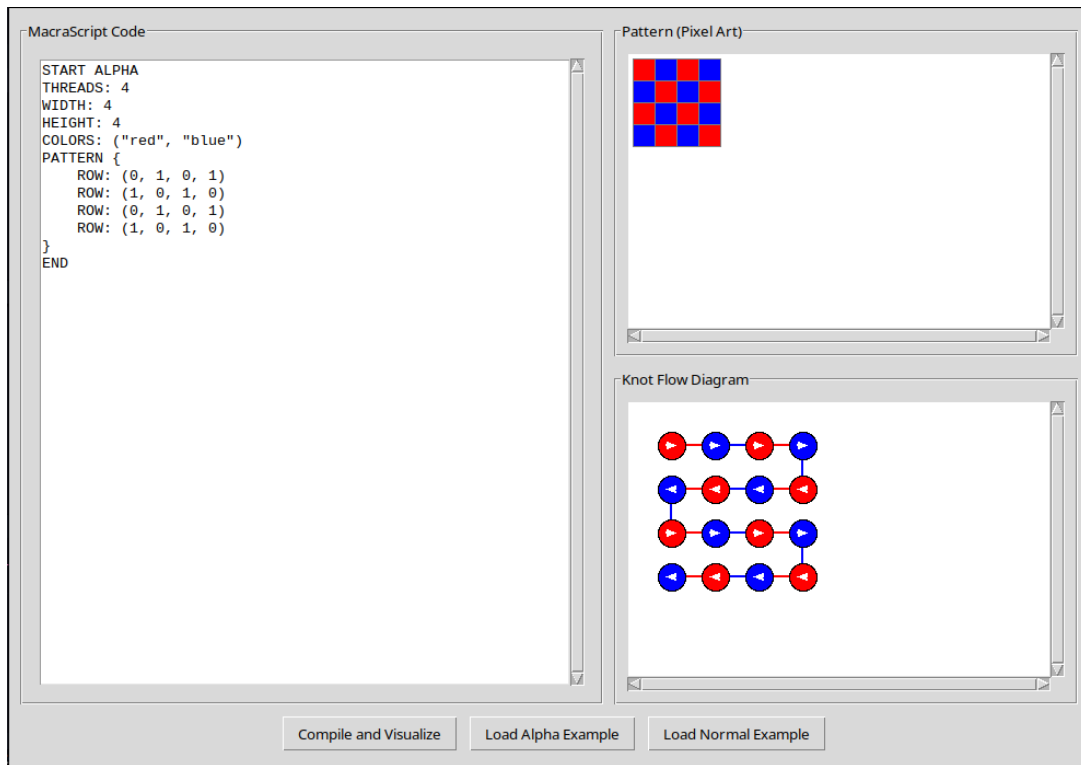


Figure 4.1: Final dual visualization for an Alpha pattern, showing the pixel-art preview and the snake-like knot-flow diagram.

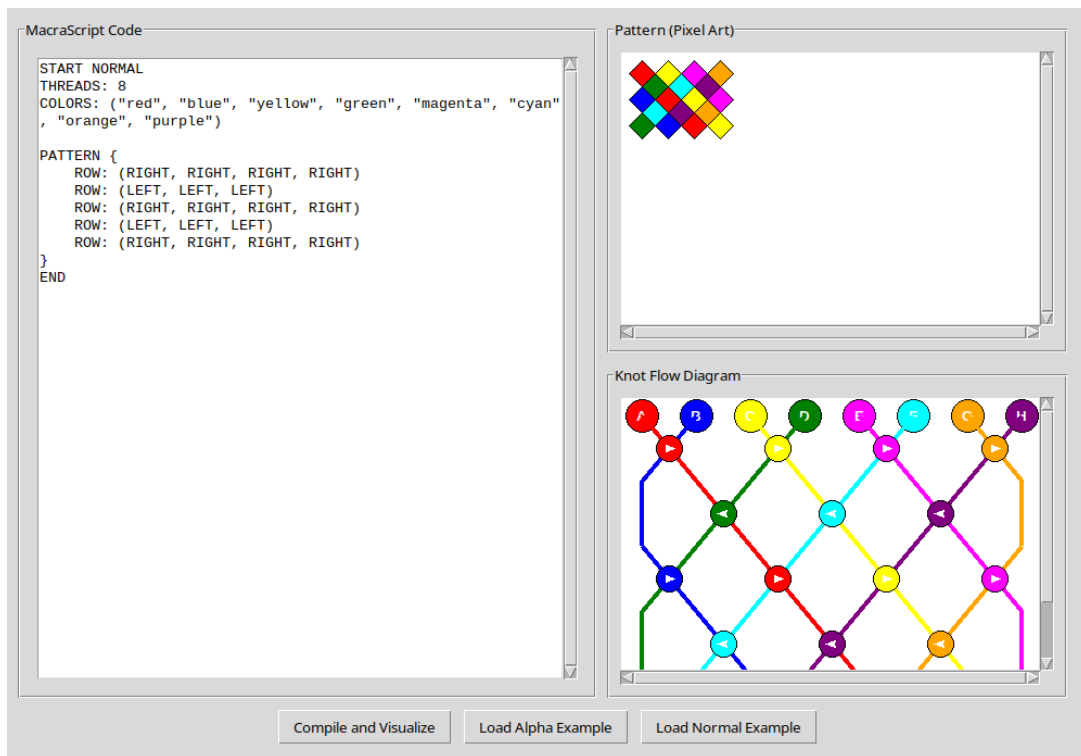


Figure 4.2: Final dual visualization for a Normal pattern, showing the diamond preview and the detailed knot-flow diagram.

4.2 Conclusion

The MacraScript project has successfully achieved its objectives. We have designed and implemented a functional DSL and compiler that significantly simplifies the process of creating and visualizing complex macramé patterns. The final system provides a robust tool that bridges the gap between abstract design and the physical craft, demonstrating the power of computational thinking in a non-traditional domain. The refined language syntax for Normal patterns proved to be a critical improvement, making the language more intuitive and powerful.

Chapter 5

Design Rationale and Reflection

5.1 Language Design Choices

The design of MacraScript was guided by the principle of simplicity and a focus on the domain’s specific needs.

- **Declarative Syntax:** We chose a declarative style over an imperative one to make the language more accessible to users without a programming background. Artisans think in terms of pattern structure, not algorithmic steps. The syntax `THREADS: 8` is more intuitive than a variable assignment like `let threads = 8;`.
- **External DSL:** An external DSL with its own compiler provides maximum flexibility. It allowed us to define a clean syntax, create a robust error-handling pipeline, and generate highly specialized graphical outputs, which would be difficult to achieve with an internal DSL hosted in a general-purpose language.
- **Abstraction in Normal Patterns:** The most significant design decision was to abstract the knot-pairing logic in `NORMAL` patterns. Manually tracking which threads form a knot in staggered rows is a major source of errors. By having the user only specify the knot’s direction (`LEFT` or `RIGHT`), we hide this complexity. The compiler handles the logic of pairing threads (1-2, 3-4 in one row; 2-3, 4-5 in the next), dramatically improving usability.

5.2 Compiler and Visualization Choices

- **Dual Graphical Output:** Providing two visualizations was crucial. The **Pixel Art View** gives an immediate, high-level preview of the final aesthetic, which is essential for design iteration. The **Knot-Flow Diagram** is a functional, step-by-step guide that is indispensable for the actual crafting process. This duality directly serves the two key phases of a macramé project: design and execution.
- **Python and Tkinter:** Python was chosen for its rapid development capabilities and extensive libraries. Tkinter, while not the most modern UI toolkit, is included in the standard library, ensuring that the compiler is easy to run on any system without complex dependencies. This aligns with the goal of making the tool accessible.

Chapter 6

Error Handling

A robust compiler must provide clear feedback. We defined a multi-level error-handling strategy that covers all phases of compilation. Errors are designed to be reported to the user in a dedicated console within the UI, as depicted in Figure 6.1.

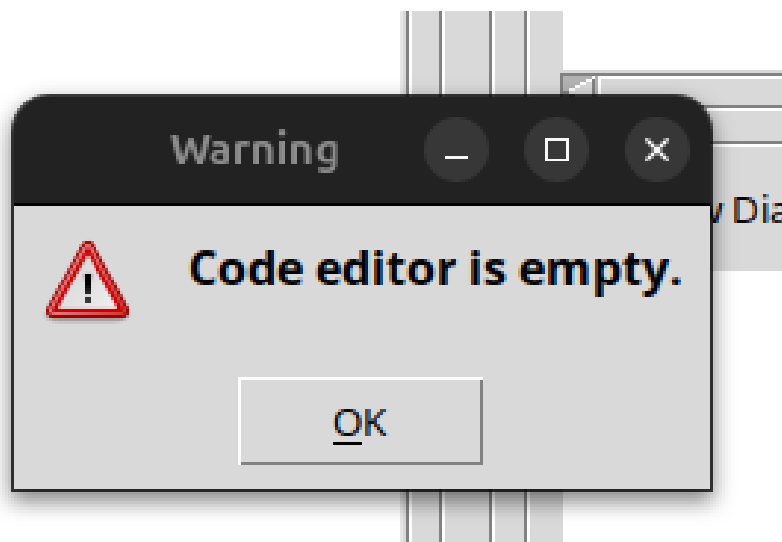


Figure 6.1: Example of error messages displayed in the UI's console.

6.1 Lexical Errors

These occur when the input contains characters not recognized by the language's alphabet.

- **Example:** Using an invalid character like @ or \$.
- **Detection:** The lexer's `MISMATCH` rule catches any character that does not match a defined token.
- **User Feedback:** Lexical Error: Unexpected character '@' at line 5, column 10.

6.2 Syntactic Errors

These occur when the code violates the language's grammar rules, even if the tokens are valid.

- **Examples:**
 - Missing a keyword (e.g., forgetting `START`).
 - Forgetting a colon, parenthesis, or brace (e.g., `THREADS 8`).
 - Incorrect order of statements (e.g., placing `COLORS` before `THREADS`).
- **Detection:** The recursive descent parser in `sintactic.py` expects specific tokens at each step. If the current token does not match, it raises a `SyntaxError`.
- **User Feedback:** Syntax Error: Expected ':' after `THREADS`, found '8' at line 2.

6.3 Semantic Errors

These occur when the code is syntactically correct but logically inconsistent with the language's rules.

- **Examples:**
 - **Inconsistent Dimensions:** Defining `HEIGHT: 5` but providing only 4 rows in an `ALPHA` pattern.
 - **Index Out of Bounds:** Using a color index of 3 when only 2 colors have been defined.
 - **Invalid Value:** Specifying `THREADS: 0`.
 - **Incorrect Knot Count:** Providing 3 knot directions in a `NORMAL` row that expects 4.
- **Detection:** The `semantic.py` analyzer performs checks after the code is parsed, validating the relationships between different parts of the program.
- **User Feedback:** Semantic Error: Row 3 has 5 colors, but pattern width is 4.

Bibliography

- [1] Fowler, M. (2010). Domain-Specific Languages. Addison-Wesley Professional.
- [2] Mernik, M., Heering, J., & Sloane, A. M. (2005). When and how to develop domain-specific languages. *ACM Computing Surveys (CSUR)*, 37(4), 316-344.
- [3] Karner, C. (2005). *Macramé: The craft of knotting*. Schiffer Publishing.